3C

DESARROLLO DE SOFTWARE

Desarrollo de aplicaciones de Software para la solución de problemas

Saber, valorar, elegir

PADRE ARRUPE
Colegio Español

DOCENTE: Daniel Sosa

COLEGIO ESPAÑOL PADRE ARRUPE

Colegio internacional.

Tercer Año de Bachillerato de Software

Desarrollo de aplicaciones de Software para la solución de problemas

Periodo 1.

GUÍA 10 – Consumo de API's con JavaFX

OBJETIVOS DE LA PRÁCTICA

Objetivo General

Desarrollar una aplicación JavaFX que consuma una API REST pública, procese datos en formato JSON y los visualice de forma estructurada y estética en una interfaz gráfica.

Objetivos Específicos

1. Comprender el funcionamiento de las APIs REST y el protocolo HTTP.
2. Implementar solicitudes HTTP utilizando la clase HttpClient de Java.
3. Procesar respuestas en formato JSON mediante la biblioteca Gson.
4. Aplicar el patrón de arquitectura MVC en una aplicación JavaFX.
5. Configurar correctamente el sistema modular de Java (module-info.java).
6. Diseñar una interfaz gráfica funcional y adaptable utilizando FXML.
7. Manejar imágenes de alta resolución dentro de contenedores visuales controlados.
8. Integrar servicios externos reales dentro de una aplicación de escritorio.

INTRODUCCIÓN TEÓRICA

El desarrollo de aplicaciones modernas requiere la integración de interfaces gráficas con servicios externos mediante el consumo de APIs REST. En este contexto, JavaFX se posiciona como una plataforma robusta para la creación de interfaces gráficas ricas (Rich Client Applications), permitiendo diseñar aplicaciones con arquitectura basada en separación de responsabilidades mediante el patrón MVC (Modelo–Vista–Controlador).

Las APIs (Application Programming Interfaces) permiten la comunicación entre sistemas a través de protocolos estandarizados como HTTP. En particular, las APIs REST utilizan solicitudes HTTP (GET, POST, etc.) para intercambiar información generalmente en formato JSON, el cual es ligero, estructurado y ampliamente utilizado en aplicaciones web y móviles.

En esta práctica se implementa el consumo de la NASA Open API, específicamente el servicio APOD (Astronomy Picture of the Day), que proporciona información diaria sobre imágenes astronómicas. La aplicación desarrollada integra:

- Cliente HTTP de Java (java.net.http)
- Procesamiento de datos en formato JSON mediante Gson
- Uso de JavaFX y FXML para diseño de interfaces
- Configuración modular con module-info.java

Además, se refuerzan conceptos clave como:

- Programación orientada a objetos

- Arquitectura por capas (Model, Service, Controller)
- Uso de reflexión en entornos modulares
- Manejo de recursos gráficos y adaptabilidad visual

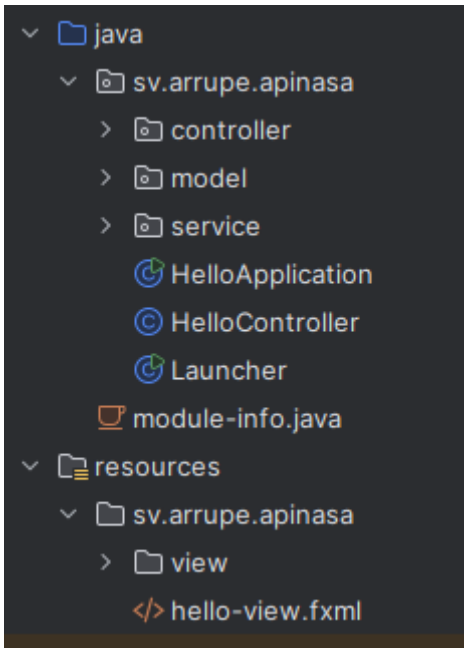
Esta práctica permite comprender cómo una aplicación de escritorio puede interactuar con servicios externos en tiempo real, fortaleciendo competencias fundamentales en el desarrollo de software moderno.

REALIZACIÓN DE PRÁCTICA

Creamos el proyecto configurándolo como en la guía anterior:

- Nombre: ApiNasa
- JavaFX
- Maven
- JDK17 o JDK21 (Preferencialmente 21)

Estructura del proyecto:



Crearemos las siguientes clases según el paquete correspondiente:

- controller
  - MainController.java
- model
  - ApodResponse.java
- service
  - NasaService.java

Dentro de resources el directorio view y la vista correspondiente que en esta ocasión será:

- main.fxml

Configuración del pom.xml

```
<dependency>
    <groupId>com.google.code.gson</groupId>
    <artifactId>gson</artifactId>
    <version>2.10.1</version>
</dependency>
```

Módulos a incorporar en module-info.java

```java
module sv.arrupe.apinasa {
    requires javafx.controls;
    requires javafx.fxml;
    requires java.net.http;
    requires com.google.gson;

    opens sv.arrupe.apinasa.controller to javafx.fxml;
    opens sv.arrupe.apinasa.model to com.google.gson;

    exports sv.arrupe.apinasa;
}
```

MainController

```java
package sv.arrupe.apinasa.controller;

import javafx.fxml.FXML;
import javafx.scene.control.*;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import sv.arrupe.apinasa.model.ApodResponse;
import sv.arrupe.apinasa.service.NasaService;

import java.time.LocalDate;

public class MainController {  1 usage

    @FXML
    private DatePicker datePicker;

    @FXML
    private Label lblTitulo;

    @FXML
    private Label lblDescripcion;

    @FXML
    private ImageView imageView;

    private NasaService nasaService = new NasaService();  1 usage

    @FXML
```

```java
    public void initialize() { datePicker.setValue(LocalDate.now()); }

    @FXML
    private void buscarImagen() {

        try {
            String fecha = datePicker.getValue().toString();

            ApodResponse apod = nasaService.getApod(fecha);

            lblTitulo.setText(apod.getTitle());
            lblDescripcion.setText(apod.getExplanation());

            if (apod.getMedia_type().equals("image")) {
                Image image = new Image(apod.getUrl(), b: true);
                imageView.setImage(image);
            } else {
                lblDescripcion.setText("El contenido es un video.\n\n"
                        + apod.getExplanation());
            }

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

ApodResponse

```java
package sv.arrupe.apinasa.model;

public class ApodResponse {  5 usages

    private String date;  1 usage
    private String explanation;  1 usage
    private String title;  1 usage
    private String url;  1 usage
    private String media_type;  1 usage

    public String getDate() { return date; }  no usages
    public String getExplanation() { return explanation; }  2 usages
    public String getTitle() { return title; }  1 usage
    public String getUrl() { return url; }  1 usage
    public String getMedia_type() { return media_type; }  1 usage
}
```

NasaService

```java
package sv.arrupe.apinasa.service;

import com.google.gson.Gson;
import sv.arrupe.apinasa.model.ApodResponse;

import java.io.IOException;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class NasaService {  3 usages

    private static final String API_KEY = "DEMO_KEY";  1 usage

    public ApodResponse getApod(String date) throws IOException, InterruptedException {  1 usage

        String url = "https://api.nasa.gov/planetary/apod?api_key="
                + API_KEY + "&date=" + date;
        HttpClient client = HttpClient.newHttpClient();
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(url))
                .GET()
                .build();

        HttpResponse<String> response =
                client.send(request, HttpResponse.BodyHandlers.ofString());

        Gson gson = new Gson();
        return gson.fromJson(response.body(), ApodResponse.class);
    }
}
```

Main.fxml

```xml
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.control.*?>
<?import javafx.scene.image.*?>
<?import javafx.scene.layout.*?>

<BorderPane xmlns:fx="http://javafx.com/fxml"
            fx:controller="sv.arrupe.apinasa.controller.MainController">

    <top>
        <VBox spacing="10" style="-fx-padding:20; -fx-background-color:#0b3d91;">
            <Label text="NASA Astronomy Picture of the Day"
                   style="-fx-text-fill:white; -fx-font-size:22; -fx-font-weight:bold;"/>

            <HBox spacing="10">
                <DatePicker fx:id="datePicker"/>
                <Button text="Buscar" onAction="#buscarImagen"/>
            </HBox>
        </VBox>
    </top>

    <center>
        <ScrollPane fitToWidth="true">
            <VBox spacing="20" style="-fx-padding:20;" alignment="TOP_CENTER">
```

```
            <Label fx:id="lblTitulo"
                   style="-fx-font-size:22; -fx-font-weight:bold;"
                   wrapText="true"/>

            <ImageView fx:id="imageView"
                       fitWidth="700"
                       fitHeight="450"
                       preserveRatio="true"
                       smooth="true"/>

            <Label fx:id="lblDescripcion"
                   wrapText="true"
                   maxWidth="700"/>

        </VBox>
    </ScrollPane>
</center>


</BorderPane>
```

Por ultimo HelloApplication.java

```
package sv.arrupe.apinasa;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Scene;
import javafx.stage.Stage;


public class HelloApplication extends Application {

    @Override
    public void start(Stage stage) throws Exception {

        FXMLLoader fxmlLoader = new FXMLLoader(
                HelloApplication.class.getResource( name: "/sv/arrupe/apinasa/view/main.fxml")
        );

        Scene scene = new Scene(fxmlLoader.load(), v: 900, v1: 700);

        stage.setTitle("NASA APOD Viewer");

        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) { launch(); }
}
```

Funcionamiento:

NOTA: Debes adquirir una API key del sitio oficial de la NASA https://api.nasa.gov/ para realizar consultas, en la aplicación de la guía se utiliza la DEMO pero tiene consultas limitadas.

ACTIVIDAD COMPLEMENTARIA

Mostrar la fecha en formato personalizado

Modificar la aplicación para que la fecha se muestre en formato:

```
12 de febrero de 2026
```

🔹 Pista: usar `DateTimeFormatter`.

Validación de fecha mínima

La API APOD no tiene datos antes de 16 de junio de 1995.

Implementar una validación que:

- Muestre un `Alert` si el usuario selecciona una fecha anterior.
- Evite realizar la solicitud HTTP.

Botón “Hoy”

Agregar un botón que:

- Coloque automáticamente la fecha actual.
- Cargue la imagen del día sin necesidad de presionar “Buscar”.

Indicador de carga (Loading)

Implementar un `ProgressIndicator` que:

- Se muestre mientras se realiza la solicitud HTTP.
- Desaparezca cuando termine.

Conceptos: `Task`, `Thread`, trabajo en segundo plano.

Manejo profesional de errores

Si ocurre:

- Error de conexión
- Error 429 (límite de API)
- Fecha inválida

Mostrar un `Alert` amigable al usuario.

Detectar cuando el contenido es video

La API a veces devuelve:

```
"media_type": "video"
```

Modificar la app para:

- Mostrar el enlace del video.
- O abrirlo en el navegador con `Desktop.getDesktop().browse()`.

Implementar caché simple

Guardar la última respuesta en memoria (Map<String, ApodResponse>).

Si el usuario busca la misma fecha:

- No hacer otra llamada HTTP.
- Usar los datos almacenados.

Separar lógica de red en un servicio independiente

Refactorizar para que:

- El controlador no tenga lógica HTTP.
- Solo llame a métodos del servicio.

Aplicar arquitectura limpia.

Implementar modo oscuro

Crear un archivo CSS alternativo:

```
dark-theme.css
```

Agregar botón que cambie dinámicamente el tema.

Guardar favoritos

Permitir:

- Marcar imagen como favorita
- Guardarla en SQLite
- Mostrar lista de favoritos

BIBLIOGRAFÍA

Google. (2023). *Gson user guide*. https://github.com/google/gson

Oracle. (2023). *The Java™ tutorials: The HTTP client API*. https://docs.oracle.com

Oracle. (2023). *JavaFX documentation*. https://openjfx.io

NASA Open APIs. (2024). *Astronomy Picture of the Day (APOD) API*. https://api.nasa.gov

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation, University of California, Irvine).

OpenJFX. (2024). *JavaFX architecture overview*. https://openjfx.io

Sosa, D., & ChatGPT. (2026). *Asistencia en el desarrollo de aplicaciones JavaFX con consumo de APIs REST*. OpenAI.